1.

a). $E[t] = \int_0^\infty t \frac{1}{\tau} e^{-\frac{t}{\tau}} dt$

$= \frac{1}{\tau}\{[-\tau t e^{-\frac{t}{\tau}}]\Big|_0^\infty + \int_0^\infty \tau e^{-\frac{t}{\tau}} dt\}$

$= \frac{1}{\tau}\{0 + \tau \int_0^\infty e^{-\frac{t}{\tau}} dt\}$

$= -\tau(0-1) = \tau$

b). $Var[t] = E[t^2] - E[t]^2$

$E[t^2] = \int_0^\infty t^2 \frac{1}{\tau} e^{-\frac{t}{\tau}} dt$

$= 2\tau^2$

$\therefore Var[t] = 2\tau^2 - \tau^2 = \tau^2$

c). $\mathcal{L}(\tau) = \prod_{i=1}^n f(t_i;\tau)$

$= \frac{1}{\tau^n} \exp(-\sum_{i=1}^n \frac{t_i}{\tau})$

$\mathcal{L}'(\tau) = \log \mathcal{L}(\tau)$

$= -n\log\tau - \frac{1}{\tau}\sum_{i=1}^n t_i$

$\frac{d}{d\tau}\mathcal{L}'(\tau) = -\frac{n}{\tau} + \frac{1}{\tau^2}\sum_{i=1}^n t_i = 0$

$\therefore n\tau = \sum_{i=1}^n t_i$

$\hat{\tau}_e = \frac{1}{n}\sum_{i=1}^n t_i$

i.e. the mean of the sample.

d). please refer to the attached notebook.

2, 3 are in notebook aswell.

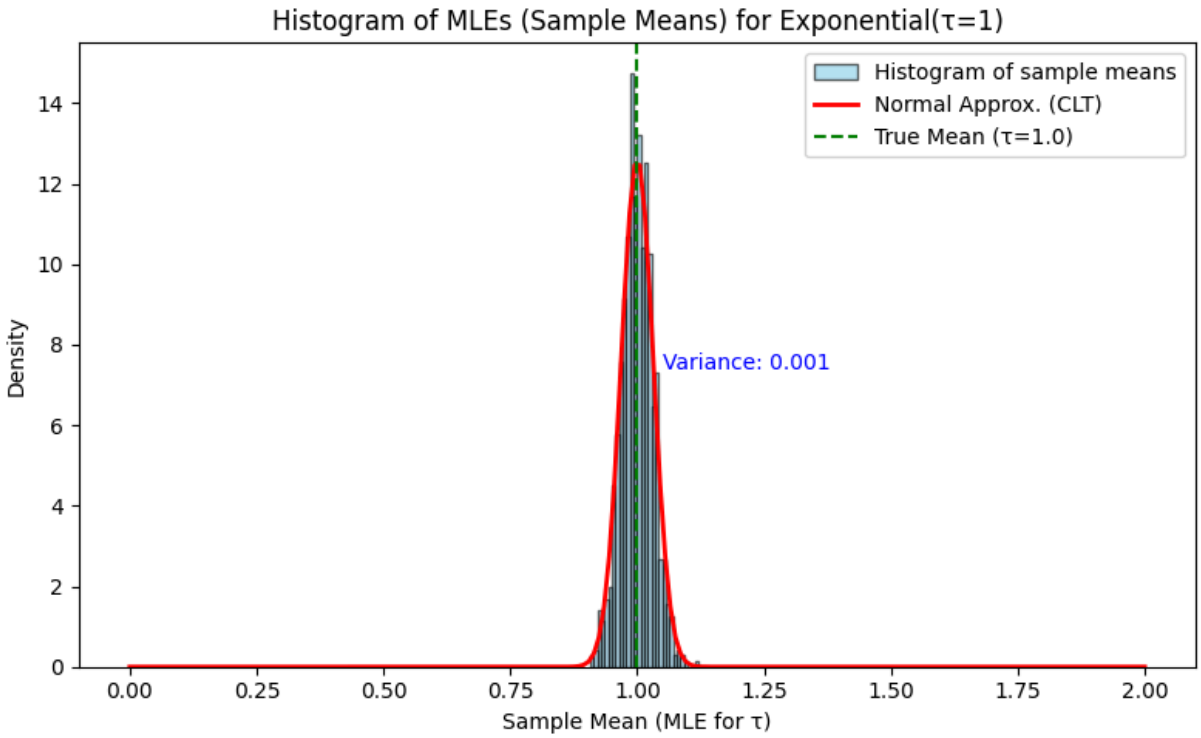
4. $Var(f) = Var(x+y)$

$= Var(x) + Var(y) + Cov(x,y) + Cov(y,x)$

$= V_{xx} + V_{xy} + V_{yx} + V_{yy}$

$\because V_{xy} = V_{yx} \quad \therefore Var(f) = V_{xx} + V_{yy} + 2V_{xy}$.

In [3]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

# --- Parameters ---
true_tau = 1.0       # True parameter of the exponential distribution
n_samples = 1000     # Number of samples in each simulated data set
n_experiments = 1000 # How many times we repeat the sampling process

# --- Step 1 & 2: Simulate data and compute sample means ---
sample_means = np.zeros(n_experiments)
for i in range(n_experiments):
    # Generate n_samples i.i.d. exponential random variables
    data = np.random.exponential(scale=true_tau, size=n_samples)

    # Compute the MLE (sample mean)
    sample_means[i] = np.mean(data)

# --- Step 3: Plot a histogram of the sample means ---
plt.figure(figsize=(8, 5))
counts, bins, _ = plt.hist(sample_means, bins=30, density=True, alpha=0.6, c
                           edgecolor='black', label='Histogram of sample mea

# --- (Optional) Overlay a Normal Approximation ---
# By the Central Limit Theorem, the mean of n exponentials should be ~ Norma
mu = true_tau
sigma = true_tau / np.sqrt(n_samples)
x = np.linspace(0, 2, 200)
pdf_normal = norm.pdf(x, loc=mu, scale=sigma)
plt.plot(x, pdf_normal, 'r-', lw=2, label='Normal Approx. (CLT)')
# --- Annotate the variance of the MLEs ---
sample_variance = np.var(sample_means)
plt.axvline(x=mu, color='g', linestyle='--', label=f'True Mean (τ={true_tau}
plt.text(mu + 0.05, max(counts) / 2, f'Variance: {sample_variance:.3f}', col
# --- Annotate and show ---
plt.title("Histogram of MLEs (Sample Means) for Exponential(τ=1)")
plt.xlabel("Sample Mean (MLE for τ)")
plt.ylabel("Density")
plt.legend()
plt.tight_layout()
plt.show()
```

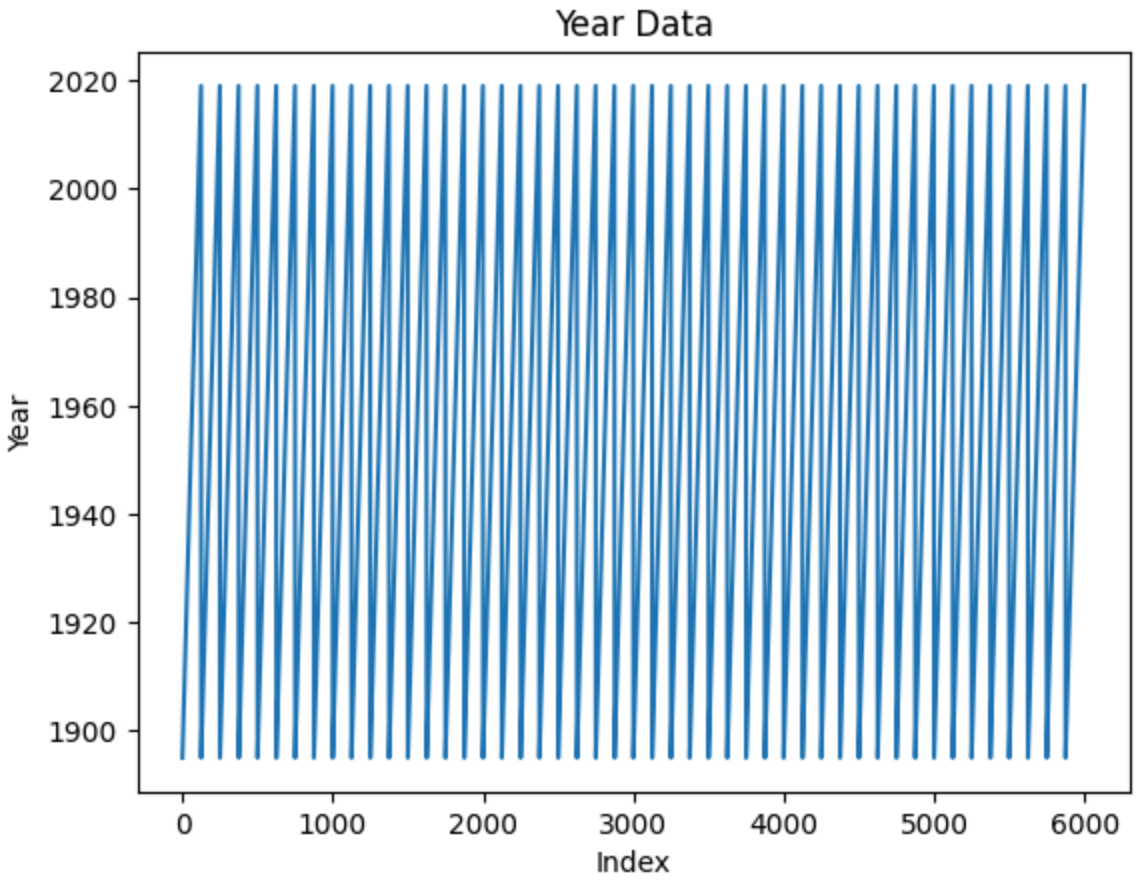

Histogram of MLEs (Sample Means) for Exponential(τ=1)
Histogram of sample means
Normal Approx. (CLT)
True Mean (τ=1.0)
Variance: 0.001
Density
Sample Mean (MLE for τ)
0
2
4
6
8
10
12
14
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00

a).

In [19]:
```
import pandas

rawData = pandas.read_csv('/Users/runzhaoguo/Documents/MQST-UCLA/402/hwk6/da
```

b).

In [20]:
```
import matplotlib.pyplot as plt

plt.plot(rawData['year'])
plt.xlabel('Index')
plt.ylabel('Year')
plt.title('Year Data')
plt.show()
```



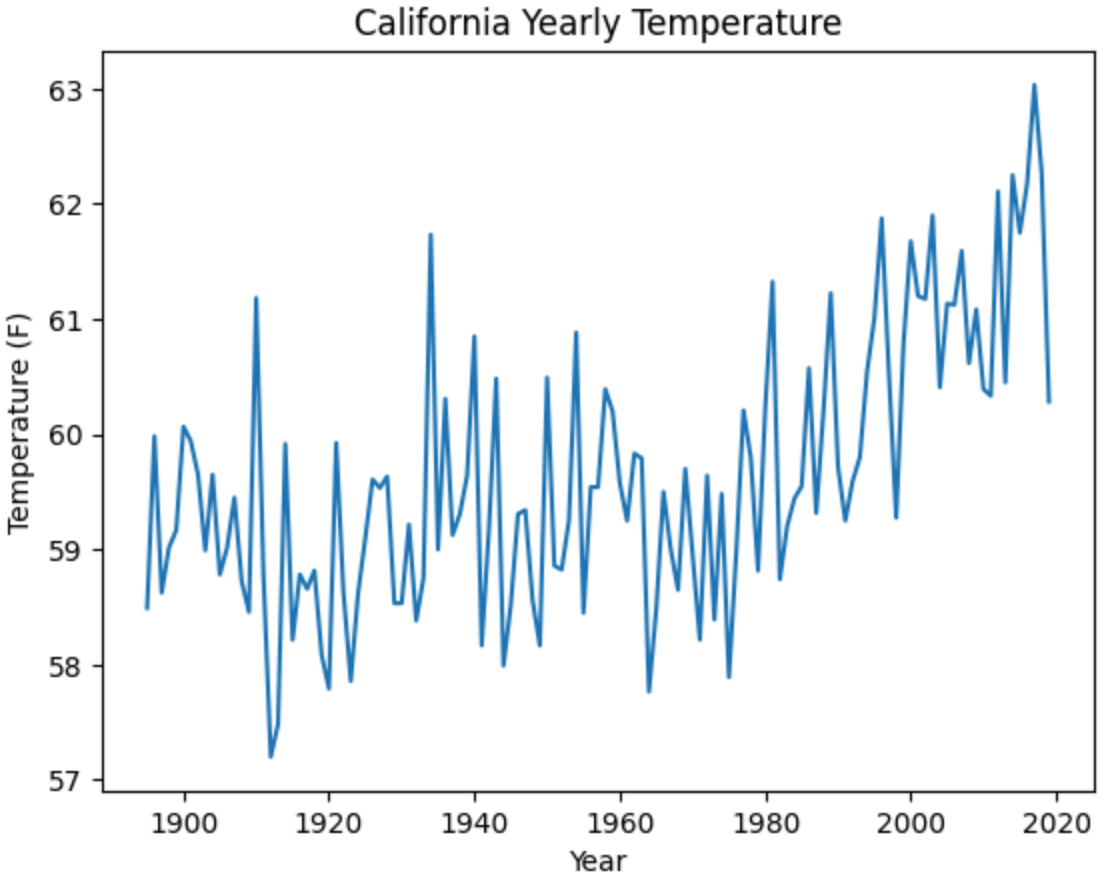
it is oscillating because this .csv file contains the data for different states(48 to be exact) concatenated together. We can se the period is 125 entries.

In [21]:
```
print(rawData['year'][0:125*10:125])
```

```
0       1895
125     1895
250     1895
375     1895
500     1895
625     1895
750     1895
875     1895
1000    1895
1125    1895
Name: year, dtype: int64
```

c).

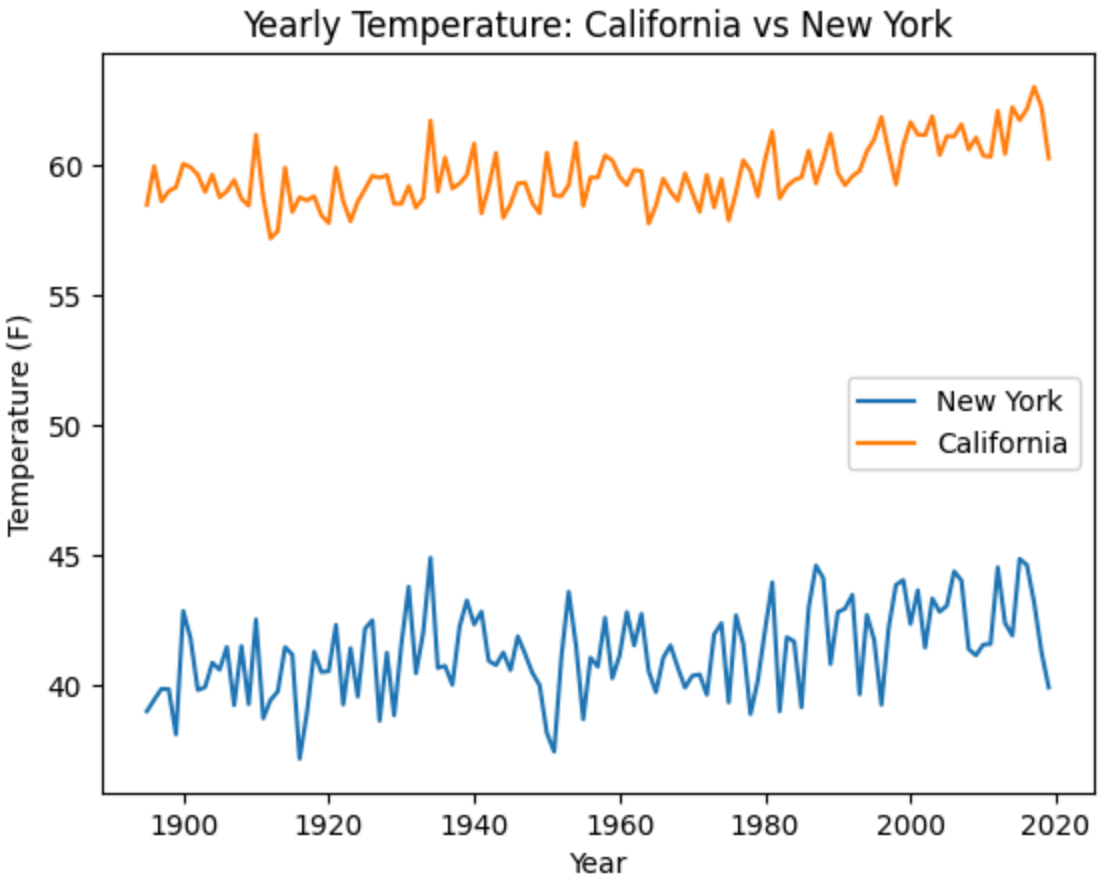
In [22]:
```
california_data = rawData[rawData['fips'] == 4]
plt.plot(california_data['year'], california_data['temp'])
plt.xlabel('Year')
plt.ylabel('Temperature (F)')
plt.title('California Yearly Temperature')
plt.show()
```



d).

In [23]:
```
# california_data = rawData[rawData['fips'] == 4]

new_york_data = rawData[rawData['fips'] == 30]
```

```
plt.plot(new_york_data['year'], new_york_data['temp'], label='New York')
plt.plot(california_data['year'], california_data['temp'], label='California')
plt.xlabel('Year')
plt.ylabel('Temperature (F)')
plt.title('Yearly Temperature: California vs New York')
plt.legend()
plt.show()
```



e).

In [24]:
```
mean_temp = rawData['temp'].mean()
std_temp = rawData['temp'].std()

print(f"Mean Temperature(all data): {mean_temp}")
print(f"Standard Deviation of Temperature(all data): {std_temp}")
```

```
Mean Temperature(all data): 51.615301388888895
Standard Deviation of Temperature(all data): 8.005998486465776
```

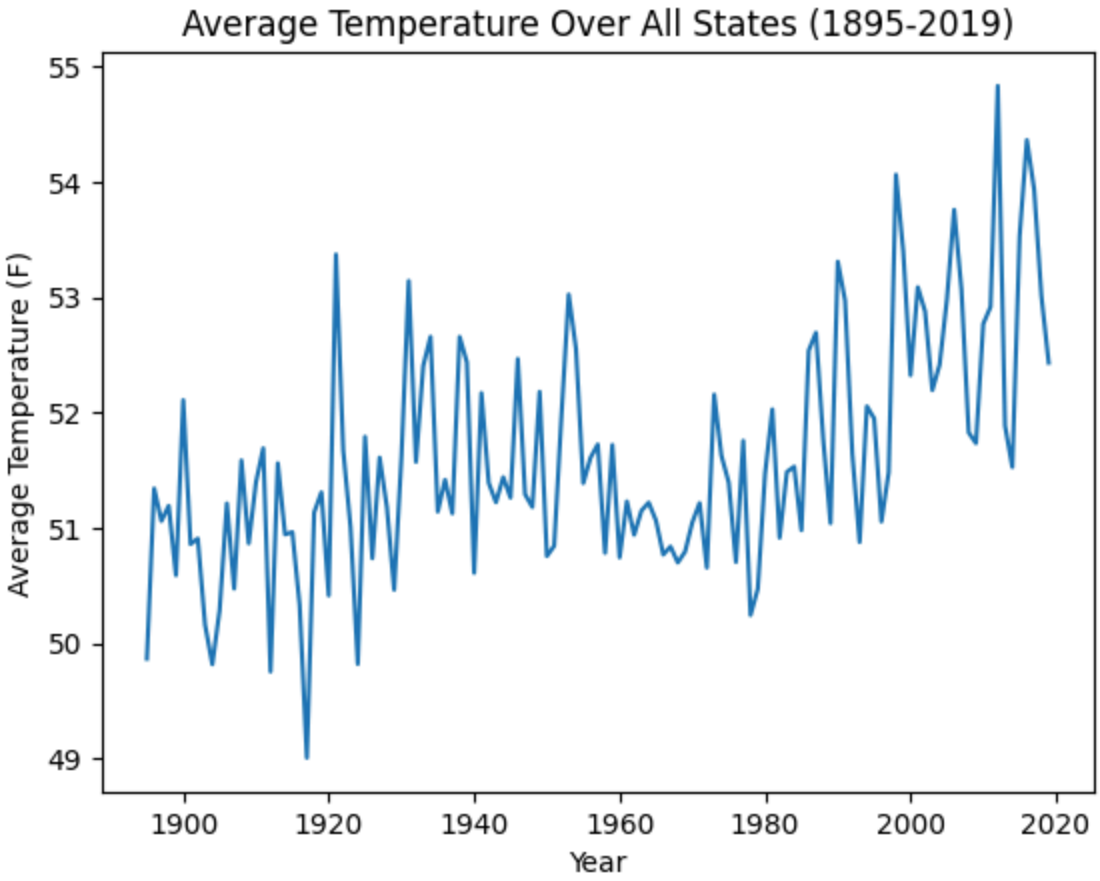
f).

In [25]:
```
mean_temp_ca = california_data['temp'].mean()
std_temp_ca = california_data['temp'].std()

print(f"Mean Temperature(CA): {mean_temp_ca}")
print(f"Standard Deviation of Temperature(CA): {std_temp_ca}")
```

```
Mean Temperature(CA): 59.65120000000001
Standard Deviation of Temperature(CA): 1.167517788617523
```

g).

In [26]:
```
# Group the data by year and calculate the mean temperature for each year
average_temp_per_year = rawData.groupby('year')['temp'].mean()

# Plot the average temperature over all states from 1895 until 2019
plt.plot(average_temp_per_year.index, average_temp_per_year.values)
plt.xlabel('Year')
plt.ylabel('Average Temperature (F)')
plt.title('Average Temperature Over All States (1895-2019)')
plt.show()
```



h).

In [27]:
```
# Filter the data for the first 30 years
first_30_years_data = rawData[rawData['year'] <= (rawData['year'].min() + 29

# Calculate the mean and standard deviation
mean_temp_first_30_years = first_30_years_data['temp'].mean()
std_temp_first_30_years = first_30_years_data['temp'].std()
```

```
print(f"Mean Temperature (First 30 Years): {mean_temp_first_30_years}")
print(f"Standard Deviation of Temperature (First 30 Years): {std_temp_first_
```

```
Mean Temperature (First 30 Years): 50.89173032407408
Standard Deviation of Temperature (First 30 Years): 8.261653852248427
```

i).

In [28]:
```
# Filter the data for the last 30 years
last_30_years_data = rawData[rawData['year'] >= (rawData['year'].max() - 29)

# Calculate the mean and standard deviation
mean_temp_last_30_years = last_30_years_data['temp'].mean()
std_temp_last_30_years = last_30_years_data['temp'].std()

print(f"Mean Temperature (Last 30 Years): {mean_temp_last_30_years}")
print(f"Standard Deviation of Temperature (Last 30 Years): {std_temp_last_30
```

```
Mean Temperature (Last 30 Years): 52.67799768518518
Standard Deviation of Temperature (Last 30 Years): 7.808442456213356
```

j).

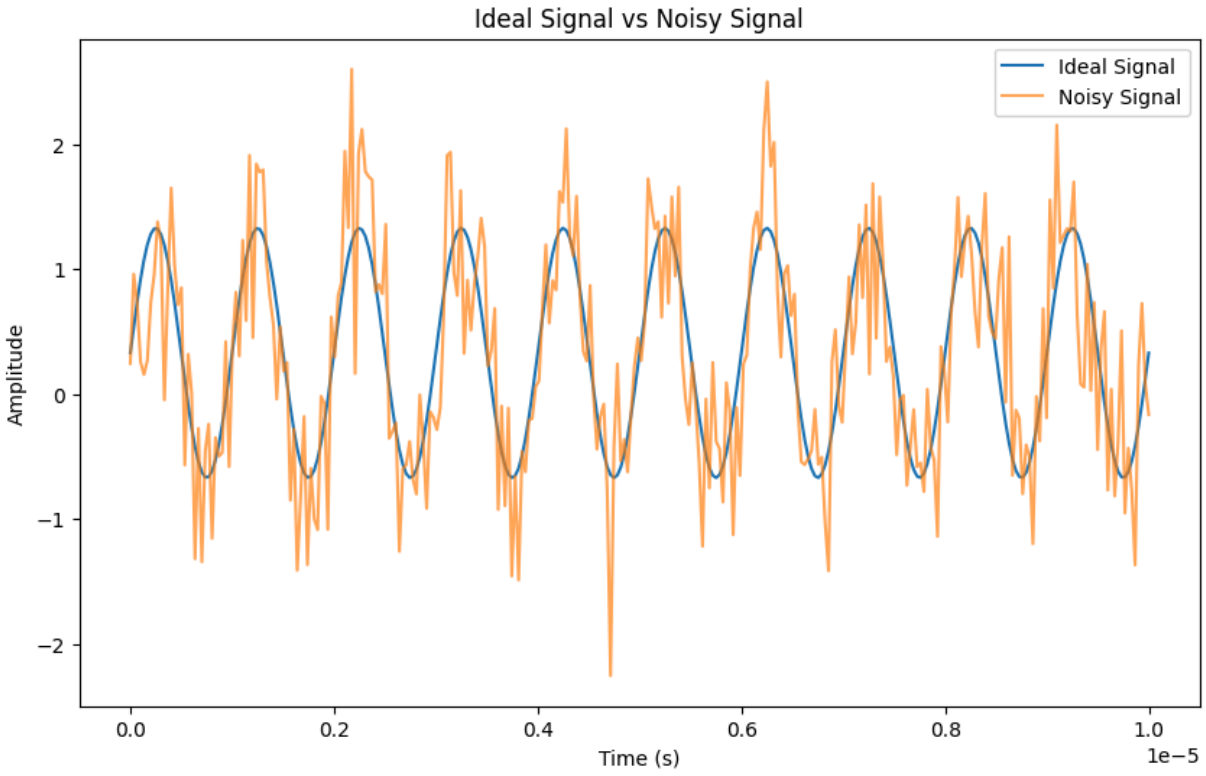
In [29]:
```
from scipy.stats import norm

# Calculate the Z-score
z_score = (mean_temp_last_30_years - mean_temp_first_30_years) / ((std_temp_

# Determine the critical Z-value for 95% and 90% confidence levels
critical_z_95 = norm.ppf(0.95)
critical_z_90 = norm.ppf(0.90)

# Print the results
print(f"Z-score: {z_score}")
print(f"Critical Z-value (95% confidence level): {critical_z_95}")
print(f"Critical Z-value (90% confidence level): {critical_z_90}")

# Conclusion
if z_score > critical_z_95:
    print("The average temperature of the last 30 years is significantly hig
elif z_score > critical_z_90:
    print("The average temperature of the last 30 years is significantly hig
else:
    print("The average temperature of the last 30 years is not significantly
```

```
Z-score: 5.96281843156864
Critical Z-value (95% confidence level): 1.644853626951472
Critical Z-value (90% confidence level): 1.2815515655446004
The average temperature of the last 30 years is significantly higher than th
e average temperature in the first 30 years at the 95% confidence level.
```

a).

In [666…

```
import numpy as np
import matplotlib.pyplot as plt


# Given parameters
A = 1.0  # Amplitude
f = 1e6  # Frequency (1 MHz)
T_total = 10e-6  # Total time duration (10 microseconds)
N = 300  # Number of samples
sigma_phase = np.pi / 10  # Standard deviation of phase noise

t = np.linspace(0, T_total, N)
omega = np.pi*2 * 10**6

ideal_signal = A*np.sin(omega * t) + 0.33

noise = np.random.normal(0, 0.5, ideal_signal.shape)
noisy_signal = ideal_signal + noise

plt.figure(figsize=(10, 6))
plt.plot(t, ideal_signal, label='Ideal Signal')
plt.plot(t, noisy_signal, label='Noisy Signal', alpha=0.7)
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.title('Ideal Signal vs Noisy Signal')
plt.legend()
plt.show()
```



b).

The standard deviation of the parameter B I expect is the standard deviation of the noisy devided by square root of the number of data points, $\frac{\sigma_{noise}}{\sqrt{N}}$, in this case is 0.0288.

In [667…

```
from scipy.optimize import curve_fit

def sin_model(t, B):
    return np.sin(omega * t) + B

initial_guess = [0.0]  # Initial guess for B
popt, pcov = curve_fit(sin_model, t, noisy_signal, p0=initial_guess)

print("The fitted value for B(Expect to be 3.3e-1):", popt[0], "\nThe standa
```

```
The fitted value for B(Expect to be 3.3e-1): 0.3185328596075806
The standard deviation(Expect to be 2.88e-2): 0.0304
```

c).

The standard deviation of B (from the fit) and the standard error of the mean (SEM) of Gaussian noise match because both are derived from the same statistical principle: averaging Gaussian noise over N samples reduces its uncertainty by a factor of $\frac{1}{\sqrt{N}}$

d).

The standard deviation for these parameters are calcuated as follows: $\sigma_A = \frac{\sigma_{noise}}{\sqrt{\frac{N}{2}}}$, $\sigma_B = \frac{\sigma_{noise}}{\sqrt{N}}$, $\sigma_f = \frac{\sigma_{noise}}{2\pi AT\sqrt{N}}$

The 95% confidence interval for parameters A, f, and B are given as :$1.96\sigma_{A/f/B}$. They are: 0.08002V, 900.50Hz, 0.05658V respectively.

The fitted parameter A and B are within reasonable CI, but the fitted f is not, I assume the reason is because f is correlated with A and B and therefore increase the deviation of f.

In [668…
```
def sin_model_d(t, A, f, B):
    return A * np.sin(np.pi*2 * f * t) + B

initial_guess = [1.0, 1e6, 0.33]  # Initial guess for B
popt, pcov = curve_fit(sin_model_d, t, noisy_signal, p0=initial_guess)

print(f"The fitted value for A(Expect to be 1):{popt[0]:.4f}\nThe fitted val
print(f"The standard deviation for A(Expect to be 4.08e-2): {np.sqrt(pcov[0]
```

```
The fitted value for A(Expect to be 1):1.0041
The fitted value for f(Expect to be 10^6):999606.4449
The fitted value for B(Expect to be 0.33): 0.3186

The standard deviation for A(Expect to be 4.08e-2): 0.0432,
The standard deviation for f(Expect to be 459.67): 1180.8278,
The standard deviation for B(Expect to be 2.88e-2): 0.0305
```

e).

The off-diagonal elements are covariance of A and f [in this case (0,1) and (1,0)], A and B [(0,2) and (2,0)], f and B [(1,2) and (2,1)]. The signs indecate these parameters correlates positively(if the covariance is positive) or negatively(if the covariance is negative). We can see in this matrix that A and f are correlated positively, while B and f are correlated negatively.

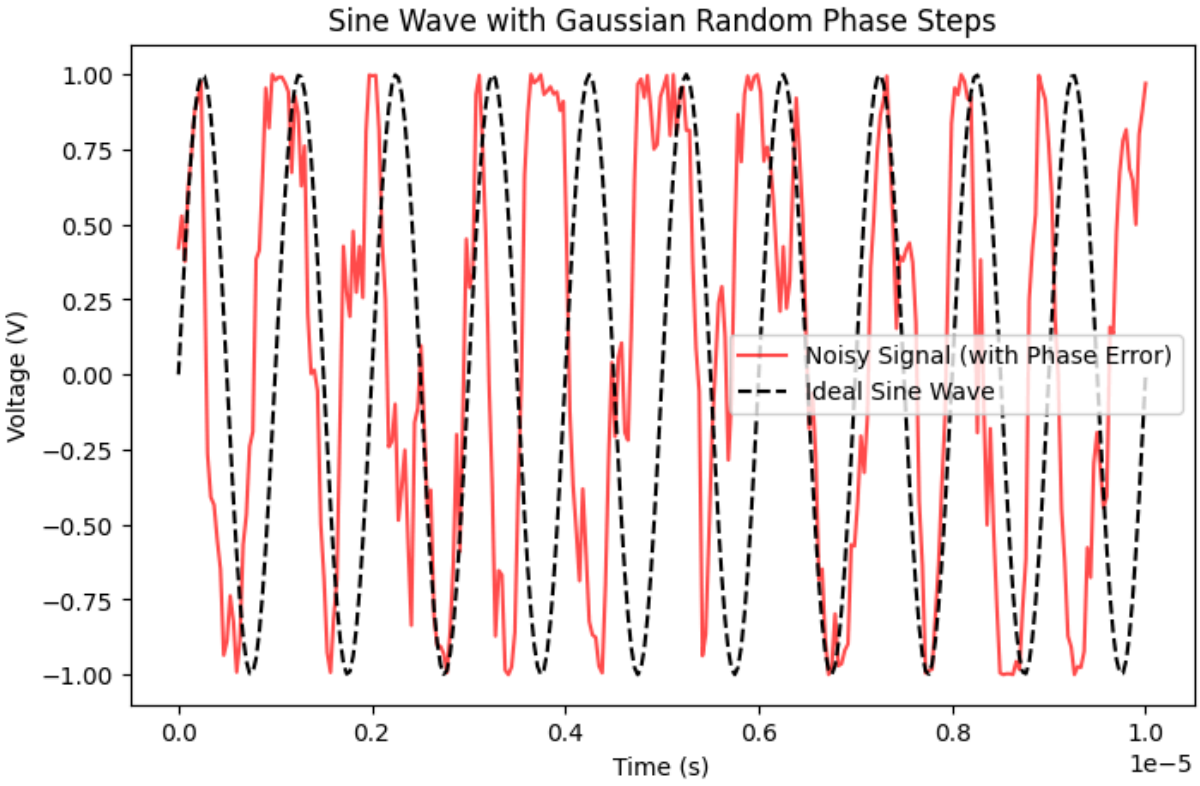
In [669…
```
print(pcov)
```

```
[[ 1.86641183e-03  6.96430740e-01  1.18191356e-08]
 [ 6.96430740e-01  1.39435423e+06 -1.12172253e-01]
 [ 1.18191356e-08 -1.12172253e-01  9.30293179e-04]]
```

f).

In [670…
```
# Generate time values
t = np.linspace(0, T_total, N)

# Generate cumulative random phase shifts
random_phase_shifts = np.random.normal(0, sigma_phase, N)  # Gaussian noise
phase_noise = np.cumsum(random_phase_shifts)  # Random walk in phase
```

```
# Generate the noisy sine wave
V_noisy = A * np.sin(2 * np.pi * f * t + phase_noise)

# Plot the results
plt.figure(figsize=(8, 5))
plt.plot(t, V_noisy, label="Noisy Signal (with Phase Error)", color="r", alp
plt.plot(t, A * np.sin(2 * np.pi * f * t), '--', label="Ideal Sine Wave", co
plt.xlabel("Time (s)")
plt.ylabel("Voltage (V)")
plt.title("Sine Wave with Gaussian Random Phase Steps")
plt.legend()
plt.show()
```



The phase-error signal is shown above, the fitted value for A and B are no longer within CI, f witnessed more error but is generally within CI. This is because the error is now accumulating with time.

In [671…

```
def sin_model_d(t, A, f, B):
    return A * np.sin(np.pi*2 * f * t) + B

initial_guess = [1.0, 1e6, 0.33]  # Initial guess for B
popt, pcov = curve_fit(sin_model_d, t, V_noisy, p0=initial_guess)

print(f"The fitted value for A(Expect to be 1):{popt[0]:.4f}\nThe fitted val
print(f"The standard deviation for A: {np.sqrt(pcov[0][0]):.4f},\nThe standa
```

```
The fitted value for A(Expect to be 1):0.3204
The fitted value for f(Expect to be 10^6):1032604.0452
The fitted value for B(Expect to be 0.33): 0.0703

The standard deviation for A: 0.0533,
The standard deviation for f: 4646.2806,
The standard deviation for B: 0.0378
```

The correlation between A,f and B,f are no longer deterministic, different relation is observed for different random singal.

In [672…
```
print(pcov)
```

```
[[ 2.84363561e-03 -1.02355604e+00 -6.74533350e-05]
 [-1.02355604e+00  2.15879239e+07 -5.39919225e+00]
 [-6.74533350e-05 -5.39919225e+00  1.43091570e-03]]
```